

智能移动开发第十次

Introduction to Coroutines in Kotlin Playground

English

1 Before you begin

2 Synchronous code

3 Asynchronous code

4 Exceptions and cancellation

5 Coroutine concepts

6 Conclusion

Simple Program

In **synchronous** code, only one conceptual task is in progress at a time. You can think of it as a sequential linear path. One task must finish completely before the next one is started. Below is an example of synchronous code.

1. Open up [Kotlin Playground](#).
2. Replace the code with the following code for a program that shows a weather forecast of sunny weather. In the `main()` function, first we print out the text: `Weather forecast`. Then we print out: `Sunny`.

```
fun main() {  
    println("Weather forecast")  
    println("Sunny")  
}
```

3. Run the code. The output from running the above code should be:

```
Weather forecast  
Sunny
```

`println()` is a synchronous call because the task of printing the text to the output is completed before execution can move to the next line of code. Because each function call in `main()` is synchronous, the entire `main()` function

Back

Next

Introduction to Coroutines in Kotlin Playground

English

1 Before you begin

2 Synchronous code

3 Asynchronous code

4 Exceptions and cancellation

5 Coroutine concepts

6 Conclusion

Add a delay

A synchronous function returns only when its task is fully complete. So after the last print statement in `main()` is executed, all work is done. The `main()` function returns and the program ends.

Now let's pretend that getting the weather forecast of sunny weather requires a network request to a remote web server. Simulate the network request by adding a delay in the code before printing that the weather forecast is sunny.

1. First, add `import kotlinx.coroutines.*` at the top of your code before the `main()` function. This imports functions you will be using from the Kotlin coroutines library.
2. Modify your code to add a call to `delay(1000)`, which delays execution of the remainder of the `main()` function by `1000` milliseconds, or 1 second. Insert this `delay()` call before the print statement for `Sunny`.

```
import kotlinx.coroutines.*  
  
fun main() {  
    println("Weather forecast")  
    delay(1000)  
    println("Sunny")  
}
```

`delay()` is actually a special **suspending function** provided by the Kotlin coroutines library. Execution of the `main()` function will suspend (or pause) at this point, and then resume once the specified duration of the delay is over (one second in this case).

Back

Next

1 Before you begin

2 Synchronous code

3 Asynchronous code

4 Exceptions and cancellation

5 Coroutine concepts

6 Conclusion

 Report a mistake

Back

```
println("Sunny")
}
```

If you run the program now, you will see the same compile error you saw earlier. A suspend function can only be called from a coroutine or another suspend function, so define `printForecast()` as a `suspend` function.

2. Add the `suspend` modifier just before the `fun` keyword in the `printForecast()` function declaration to make it a suspending function.

```
import kotlinx.coroutines.*

fun main() {
    runBlocking {
        println("Weather forecast")
        printForecast()
    }
}

suspend fun printForecast() {
    delay(1000)
    println("Sunny")
}
```



Remember that `delay()` is a suspending function, and now you've made `printForecast()` a suspending function

Next

1 Before you begin

2 Synchronous code

3 Asynchronous code

4 Exceptions and cancellation

5 Coroutine concepts

6 Conclusion

 Report a mistake

Back

`{ printTemperature() }` also returns even before all work is completed.

3. (Optional) If you want to see how much faster the program is now, you could add the `measureTimeMillis()` code to check the execution time.

```
import kotlin.system.*
import kotlinx.coroutines.*

fun main() {
    val time = measureTimeMillis {
        runBlocking {
            println("Weather forecast")
            launch {
                printForecast()
            }
            launch {
                printTemperature()
            }
        }
        println("Execution time: ${time / 1000.0} seconds")
    }
    ...
}
```



Output:

Next

1 Before you begin

2 Synchronous code

3 Asynchronous code

4 Exceptions and cancellation

5 Coroutine concepts

6 Conclusion

 Report a mistake

Back

data. Update the function names from `printForecast()` and `printTemperature()` to `getForecast()` and `getTemperature()`.

```
...  
  
suspend fun getForecast(): String {  
    delay(1000)  
    return "Sunny"  
}  
  
suspend fun getTemperature(): String {  
    delay(1000)  
    return "30\u00b0C"  
}
```

2. Modify your `runBlocking()` code so that it uses `async()` instead of `launch()` for the two coroutines. Store the return value of each `async()` call in variables called `forecast` and `temperature`, which are `Deferred` objects that hold a result of type `String`. (Specifying the type is optional because of type inference in Kotlin, but it's included below so you can more clearly see what's being returned by the `async()` calls.)

```
import kotlinx.coroutines.*  
  
fun main() {  
    runBlocking {  
        ...  
    }  
}
```

Next

1 Before you begin

2 Synchronous code

3 Asynchronous code

4 Exceptions and cancellation

5 Coroutine concepts

6 Conclusion

 Report a mistake

Back

Lastly, the error gets propagated upwards and the program crashes with the `IllegalArgumentException`.

Try-catch exceptions

If you know that certain parts of your code can possibly throw an exception, then you can surround that code with a [try-catch](#) block. You can catch the exception and handle it more gracefully in your app, such as by showing the user a helpful error message. Here's a code snippet of how it might look:

```
try {  
    // Some code that may throw an exception  
} catch (e: IllegalArgumentException) {  
    // Handle exception  
}
```

This approach also works for asynchronous code with coroutines. You can still use a try-catch expression to catch and handle exceptions in coroutines. The reason is because with structured concurrency, the sequential code is still synchronous code so the try-catch block will still work in the same expected way.

```
...  
  
fun main() {  
    runBlocking {  
        ...  
        try {  
            ...  
        }  
    }  
}
```

Next

- 1 Before you begin
- 2 Synchronous code
- 3 Asynchronous code
- 4 Exceptions and cancellation
- 5 **Coroutine concepts**
- 6 Conclusion

[Report a mistake](#)

Back

10 results found.

4. Add print statements to see what thread you are on by calling `Thread.currentThread().name`.

```
import kotlinx.coroutines.*

fun main() {
    runBlocking {
        println("${Thread.currentThread().name} - runBlocking function")
        launch {
            println("${Thread.currentThread().name} - launch function")
            withContext(Dispatchers.Default) {
                println("${Thread.currentThread().name} - withContext function")
                delay(1000)
                println("10 results found.")
            }
            println("${Thread.currentThread().name} - end of launch function")
        }
        println("Loading...")
    }
}
```

5. Run the program. The output should be:

Next

- 1 Before you begin
- 2 App overview
- 3 **Implement race progress**
- 4 Start the race
- 5 Structured concurrency
- 6 Launch concurrent tasks
- 7 Write unit tests to test coroutines
- 8 Get the solution code

[Report a mistake](#)

Back

You need the `run()` function which compares the player's `currentProgress` with the `maxProgress`, which reflects the total progress of the race, and uses the `delay()` suspend function to add a slight delay between progress increments. This function must be a `suspend` function since it is calling another suspend function `delay()`. Also, you will call this function later in the codelab from a coroutine. Follow these steps to implement the function:

1. Open `RaceParticipant` class, which is part of the starter code.
2. Inside the `RaceParticipant` class, define a new `suspend` function named `run()`.

```
class RaceParticipant(
    ...
) {
    var currentProgress by mutableStateOf(initialProgress)
    private set

    suspend fun run() {
        ...
    }
}
```

3. To simulate the progress of the race, add a `while` loop that runs until `currentProgress` reaches the value `maxProgress`, which is set to `100`.

Next

Next step

1 Before you begin

2 App overview

3 Implement race progress

4 Start the race

5 Structured concurrency

6 Launch concurrent tasks

7 Write unit tests to test coroutines

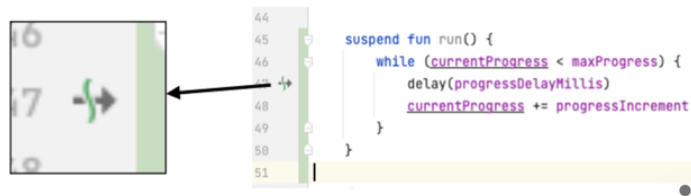
8 Get the solution code

 Report a mistake

Back

}

When you look at the code you just added, you will see an icon on the left of the call to the `delay()` function in Android Studio, as shown in the screenshot below:



This icon indicates the suspension point when the function might suspend and resume again later.

The main thread is not blocked while the coroutine is waiting to complete the delay duration, as shown in the following diagram:



Next

Run a race



Player 1



75%

100%

Player 2



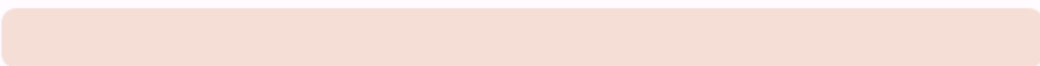
0%

100%

Pause

Reset

Player 1



0%

100%

Player 2



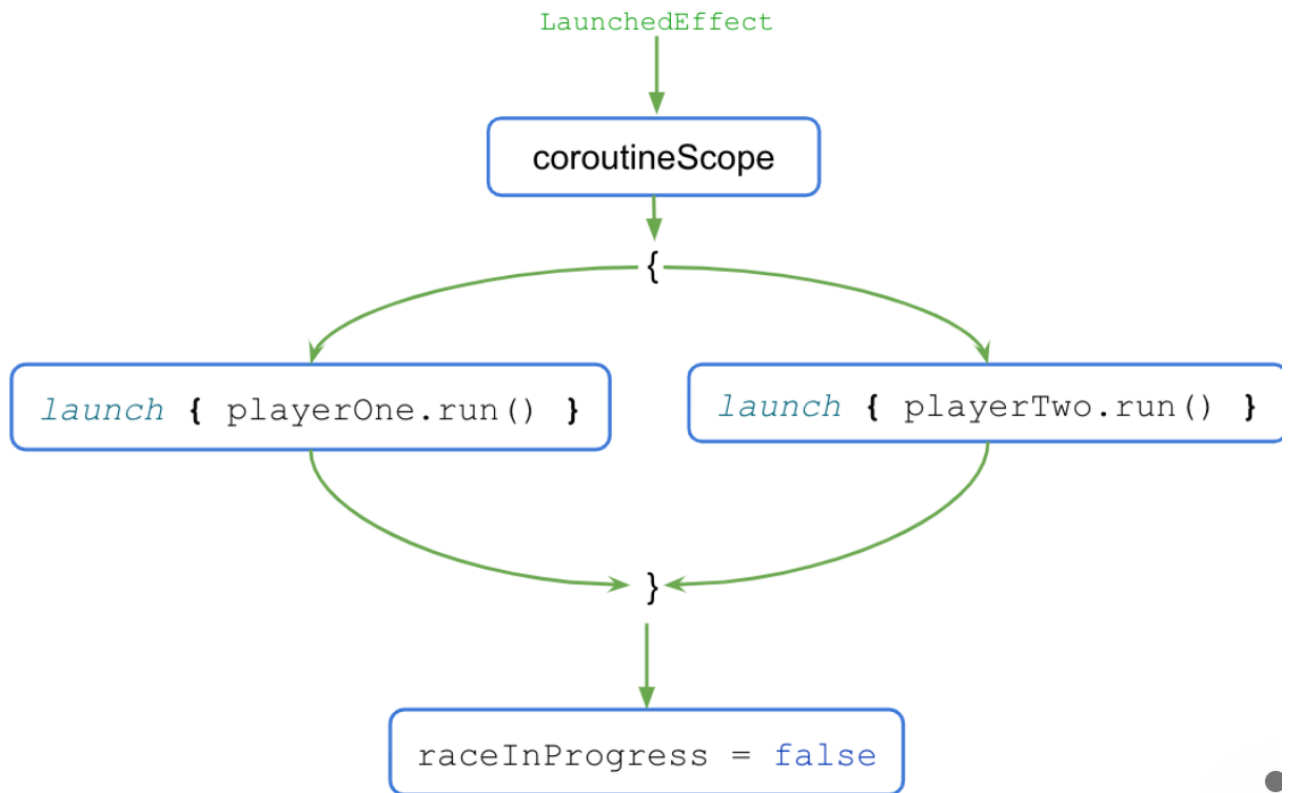
0%

100%

Start

Reset

The execution flow is shown in the following diagram:



Get data from the internet

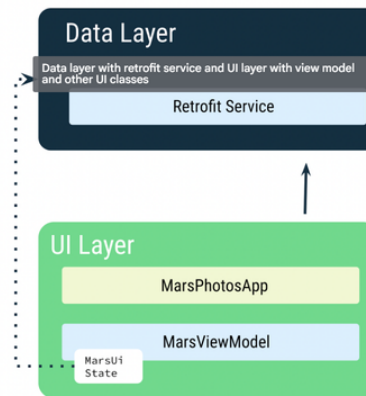
English

- 1 Before you begin
- 2 App overview
- 3 Explore the Mars Photos starter app
- 4 Introduction to web services
- 5 Web services and Retrofit
- 6 Connecting to the Internet
- 7 Add Internet permission and Exception Handling
- 8 Parse the JSON response with Kotlin serialization

[Report a mistake](#)

Back

In this codelab, you create a layer for the network service that communicates with the backend server and fetches the required data. You use a third-party library, called [Retrofit](#), to implement this task. You learn more about this later. The `ViewModel1` communicates with the data layer, and the rest of the app is transparent to this implementation.



The `MarsViewModel1` is responsible for making the network call to get the Mars photos data. In the `ViewModel1`, you use `MutableState` to update the app UI when the data changes.

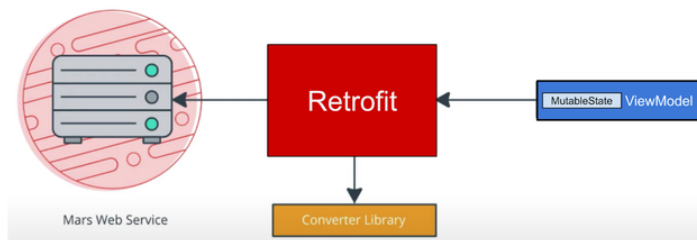
Next

- 1 Before you begin
 - 2 App overview
 - 3 Explore the Mars Photos starter app
 - 4 Introduction to web services
 - 5 Web services and Retrofit
 - 6 Connecting to the Internet
 - 7 Add Internet permission and Exception Handling
 - 8 Parse the JSON response with Kotlin serialization
- [Report a mistake](#)

6. Connecting to the Internet

You use the Retrofit library to talk to the Mars web service and display the raw JSON response as a `String`. The placeholder `Text` either displays the returned JSON response string or a message indicating a connection error.

Retrofit creates a network API for the app based on the content from the web service. It fetches data from the web service and routes it through a separate converter library that knows how to decode the data and return it in the form of objects, like `String`. Retrofit includes built-in support for popular data formats, such as XML and JSON. Retrofit ultimately creates the code to call and consume this service for you, including critical details, such as running the requests on background threads.

[Back](#)[Next](#)

- 1 Before you begin
 - 2 App overview
 - 3 Explore the Mars Photos starter app
 - 4 Introduction to web services
 - 5 Web services and Retrofit
 - 6 Connecting to the Internet
 - 7 Add Internet permission and Exception Handling
 - 8 Parse the JSON response with Kotlin serialization
- [Report a mistake](#)

3. Run the app one more time. Notice that the app does not crash this time.

Add State UI

In the `MarsViewModel` class, the status of the most recent web request, `marsUiState`, is saved as a mutable state object. However, this class lacks the ability to save the different status: loading, success, and failure.

- **Loading** status indicates the app is waiting for data.
- **Success** status indicates the data was successfully retrieved from the web service.
- **Error** status indicates any network or connection errors.

To represent these three states in your application, you use a *sealed interface*. A *sealed interface* makes it easy to manage state by limiting the possible values. In the Mars Photos app, you restrict the `marsUiState` web response to three states (data class objects): loading, success, and error, which looks like the following code:

```
// No need to copy over
sealed interface MarsUiState {
    data class Success : MarsUiState
    data class Loading : MarsUiState
    data class Error : MarsUiState
}
```

In the above code snippet, in the case of a successful response, you receive Mars photo information from the server. In

[Back](#)[Next](#)[Next step](#)